

✓ PyTorch task: Neural Network Classification

```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

FOLDERNAME = 'intro_to_ai/assignments/assignment1'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load

# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/intro_to_ai/datasets/
# !bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME

Mounted at /content/drive
/content/drive/My Drive/intro_to_ai/assignments/assignment1/intro_to_ai/datasets
/content/drive/My Drive/intro_to_ai/assignments/assignment1
```

Table of Contents

This assignment has 5 parts. You will learn PyTorch on **three different levels of abstraction**, which will help you understand it better and prepare you for the final project.

1. Part I, Preparation: we will use CIFAR-10 dataset.
2. Part II, Barebones PyTorch: **Abstraction level 1**, we will work directly with the lowest-level PyTorch Tensors.
3. Part III, PyTorch Module API: **Abstraction level 2**, we will use `nn.Module` to define arbitrary neural network architecture.
4. Part IV, PyTorch Sequential API: **Abstraction level 3**, we will use `nn.Sequential` to define a linear feed-forward network very conveniently.
5. Part V, CIFAR-10 open-ended challenge: please implement your own network to get as high accuracy as possible on CIFAR-10. You can experiment with any layer, optimizer, hyperparameters or other advanced features.

Here is a table of comparison:

API	Flexibility	Convenience
Barebone	High	Low
<code>nn.Module</code>	High	Medium
<code>nn.Sequential</code>	Low	High

✓ GPU

You can manually switch to a GPU device on Colab by clicking `Runtime -> Change runtime type` and selecting `GPU` under `Hardware Accelerator`. You should do this before running the following cells to import packages, since the kernel gets restarted upon switching runtimes.

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torch.utils.data import sampler

import torchvision.datasets as dset
import torchvision.transforms as T

import numpy as np

USE_GPU = True
dtype = torch.float32 # We will be using float throughout this assignment.

if USE_GPU and torch.cuda.is_available():
    device = torch.device('cuda')
else:
    device = torch.device('cpu')

# Constant to control how frequently we print train loss.
print_every = 100
print('using device:', device)

using device: cuda

```

✓ Part I. Preparation

Now, let's load the CIFAR-10 dataset. This might take a couple of minutes the first time you run the code, but the files should stay cached afterwards.

In earlier sections of this assignment, dataset handling was performed manually. Here, we use PyTorch's built-in utilities to load, preprocess, and iterate over the data in minibatches.

```

NUM_TRAIN = 49000

# The torchvision.transforms package provides tools for preprocessing data.
# Here we set up a transform that normalizes the data by subtracting the mean
# RGB value and dividing by the standard deviation of each RGB channel.
# The mean and standard deviation values are hardcoded.
transform = T.Compose([
    T.ToTensor(),
    T.Normalize((0.4914, 0.4822, 0.4465),
                (0.2023, 0.1994, 0.2010))
])

# We set up a Dataset object for each split (train / val / test).
# Datasets load examples one at a time, so we wrap each Dataset in a DataLoader
# which forms minibatches. The training set is split into train and validation
# subsets using a Sampler.
cifar10_train = dset.CIFAR10('./intro_to_ai/datasets', train=True, download=True,
                             transform=transform)
loader_train = DataLoader(cifar10_train, batch_size=64,
                          sampler=sampler.SubsetRandomSampler(range(NUM_TRAIN)))

cifar10_val = dset.CIFAR10('./intro_to_ai/datasets', train=True, download=True,
                           transform=transform)
loader_val = DataLoader(cifar10_val, batch_size=64,
                       sampler=sampler.SubsetRandomSampler(range(NUM_TRAIN, 50000)))

cifar10_test = dset.CIFAR10('./intro_to_ai/datasets', train=False, download=True,
                             transform=transform)
loader_test = DataLoader(cifar10_test, batch_size=64)

```

Part II. Barebones PyTorch

PyTorch ships with high-level APIs to help us define model architectures conveniently, which we will cover in Part II of this assignment. In this section, we will start with the barebone PyTorch elements to understand the autograd engine better. After this exercise, you will come to appreciate the high-level model API more.

We will start with a simple fully-connected ReLU network with two hidden layers and no biases for CIFAR classification. This implementation computes the forward pass using operations on PyTorch Tensors, and uses PyTorch autograd to compute gradients. It is important that you understand every line, because you will write a harder version after the example.

When we create a PyTorch Tensor with `requires_grad=True`, then operations involving that Tensor will not just compute values; they will also build up a computational graph in the background, allowing us to easily backpropagate through the graph to compute gradients of some Tensors with respect to a downstream loss. Concretely if `x` is a Tensor with `x.requires_grad == True` then after backpropagation `x.grad` will be another Tensor holding the gradient of `x` with respect to the scalar loss at the end.

PyTorch Tensors: Flatten Function

A PyTorch Tensor is conceptionally similar to a numpy array: it is an n -dimensional grid of numbers, and like numpy PyTorch provides many functions to efficiently operate on Tensors. As a simple example, we provide a `flatten` function below which reshapes image data for use in a fully-connected neural network.

Recall that image data is typically stored in a Tensor of shape $N \times C \times H \times W$, where:

- N is the number of datapoints
- C is the number of channels
- H is the height of the intermediate feature map in pixels
- W is the width of the intermediate feature map in pixels

When we use fully connected affine layers to process the image, however, we want each datapoint to be represented by a single vector -- it's no longer useful to segregate the different channels, rows, and columns of the data. So, we use a "flatten" operation to collapse the $C \times H \times W$ values per representation into a single long vector. The `flatten` function below first reads in the N , C , H , and W values from a given batch of data, and then returns a "view" of that data. "View" is analogous to numpy's "reshape" method: it reshapes `x`'s dimensions to be $N \times ??$, where $??$ is allowed to be anything (in this case, it will be $C \times H \times W$, but we don't need to specify that explicitly).

```
def flatten(x):
    N = x.shape[0] # read in N, C, H, W
    return x.view(N, -1) # "flatten" the C * H * W values into a single vector per image
```

```
def test_flatten():
    x = torch.arange(12).view(2, 1, 3, 2)
    print('Before flattening: ', x)
    print('After flattening: ', flatten(x))
```

```
test_flatten()
```

```
Before flattening:  tensor([[[[ 0,  1],
      [ 2,  3],
      [ 4,  5]]],
      [[[ 6,  7],
      [ 8,  9],
      [10, 11]]]])
After flattening:  tensor([[ 0,  1,  2,  3,  4,  5],
      [ 6,  7,  8,  9, 10, 11]])
```

✓ Barebones PyTorch: Two-Layer Network

Here we define a function `two_layer_fc` which performs the forward pass of a two-layer fully-connected ReLU network on a batch of image data. After defining the forward pass we check that it doesn't crash and that it produces outputs of the right shape by running zeros through the network.

You don't have to write any code here, but it's important that you read and understand the implementation.

```
import torch.nn.functional as F # useful stateless functions

def two_layer_fc(x, params):
    """
    A fully-connected neural network; the architecture is:
    NN is fully connected -> ReLU -> fully connected layer.
    Note that this function only defines the forward pass;
    PyTorch will take care of the backward pass for us.

    The input to the network will be a minibatch of data, of shape
    (N, d1, ..., dM) where  $d1 * \dots * dM = D$ . The hidden layer will have H units,
    and the output layer will produce scores for C classes.

    Inputs:
    - x: A PyTorch Tensor of shape (N, d1, ..., dM) giving a minibatch of
      input data.
    - params: A list [w1, w2] of PyTorch Tensors giving weights for the network;
      w1 has shape (D, H) and w2 has shape (H, C).

    Returns:
    - scores: A PyTorch Tensor of shape (N, C) giving classification scores for
      the input data x.
    """
    # first we flatten the image
    x = flatten(x) # shape: [batch_size, C x H x W]

    w1, w2 = params

    # Forward pass: compute predicted y using operations on Tensors. Since w1 and
    # w2 have requires_grad=True, operations involving these Tensors will cause
    # PyTorch to build a computational graph, allowing automatic computation of
    # gradients. Since we are no longer implementing the backward pass by hand we
    # don't need to keep references to intermediate values.
    # you can also use `.clamp(min=0)`, equivalent to F.relu()
    x = F.relu(x.mm(w1))
    x = x.mm(w2)
    return x

def two_layer_fc_test():
    hidden_layer_size = 42
    x = torch.zeros((64, 50), dtype=dtype) # minibatch size 64, feature dimension 50
    w1 = torch.zeros((50, hidden_layer_size), dtype=dtype)
    w2 = torch.zeros((hidden_layer_size, 10), dtype=dtype)
    scores = two_layer_fc(x, [w1, w2])
    print(scores.size()) # you should see [64, 10]

two_layer_fc_test()

torch.Size([64, 10])
```

✓ Barebones PyTorch: Initialization

Let's write a couple utility methods to initialize the weight matrices for our models.

- `random_weight(shape)` initializes a weight tensor with the Kaiming normalization method.
- `zero_weight(shape)` initializes a weight tensor with all zeros. Useful for instantiating bias parameters.

The `random_weight` function uses the Kaiming normal initialization method, described in:

He et al, *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*, ICCV 2015, <https://arxiv.org/abs/1502.01852>

```
def random_weight(shape):
    """
    Create random Tensors for weights; setting requires_grad=True means that we
    want to compute gradients for these Tensors during the backward pass.
    We use Kaiming normalization: sqrt(2 / fan_in)
    """
    if len(shape) == 2: # FC weight
        fan_in = shape[0]
    else:
        fan_in = np.prod(shape[1:]) # general fan-in computation
    # randn is standard normal distribution generator.
    w = torch.randn(shape, device=device, dtype=dtype) * np.sqrt(2. / fan_in)
    w.requires_grad = True
    return w

def zero_weight(shape):
    return torch.zeros(shape, device=device, dtype=dtype, requires_grad=True)

# create a weight of shape [3 x 5]
# you should see the type `torch.cuda.FloatTensor` if you use GPU.
# Otherwise it should be `torch.FloatTensor`
random_weight((3, 5))

tensor([[ -1.2829,  0.5756,  0.1109,  1.4707,  0.5452],
        [ -0.5886, -0.1494, -0.1110,  1.1924,  2.3100],
        [ -1.7106,  0.5646, -0.0451, -0.5410, -0.8892]], device='cuda:0',
        requires_grad=True)
```

✓ Barebones PyTorch: Check Accuracy

When training the model we will use the following function to check the accuracy of our model on the training or validation sets.

When checking accuracy we don't need to compute any gradients; as a result we don't need PyTorch to build a computational graph for us when we compute scores. To prevent a graph from being built we scope our computation under a `torch.no_grad()` context manager.

```
def check_accuracy_part2(loader, model_fn, params):
    """
    Check the accuracy of a classification model.

    Inputs:
    - loader: A DataLoader for the data split we want to check
    - model_fn: A function that performs the forward pass of the model,
      with the signature scores = model_fn(x, params)
    - params: List of PyTorch Tensors giving parameters of the model

    Returns: Nothing, but prints the accuracy of the model
    """
    split = 'val' if loader.dataset.train else 'test'
    print('Checking accuracy on the %s set' % split)
    num_correct, num_samples = 0, 0
    with torch.no_grad():
        for x, y in loader:
            x = x.to(device=device, dtype=dtype) # move to device, e.g. GPU
            y = y.to(device=device, dtype=torch.int64)
            scores = model_fn(x, params)
            _, preds = scores.max(1)
            num_correct += (preds == y).sum()
```

```

num_samples += preds.size(0)
acc = float(num_correct) / num_samples
print('Got %d / %d correct (%.2f%%)' % (num_correct, num_samples, 100 * acc))

```

✓ BareBones PyTorch: Training Loop

We can now set up a basic training loop to train our network. We will train the model using stochastic gradient descent without momentum. We will use `torch.functional.cross_entropy` to compute the loss; you can [read about it here](#).

The training loop takes as input the neural network function, a list of initialized parameters (`[w1, w2]` in our example), and learning rate.

```

def train_part2(model_fn, params, learning_rate):
    """
    Train a model on CIFAR-10.

    Inputs:
    - model_fn: A Python function that performs the forward pass of the model.
      It should have the signature scores = model_fn(x, params) where x is a
      PyTorch Tensor of image data, params is a list of PyTorch Tensors giving
      model weights, and scores is a PyTorch Tensor of shape (N, C) giving
      scores for the elements in x.
    - params: List of PyTorch Tensors giving weights for the model
    - learning_rate: Python scalar giving the learning rate to use for SGD

    Returns: Nothing
    """
    for t, (x, y) in enumerate(loader_train):
        # Move the data to the proper device (GPU or CPU)
        x = x.to(device=device, dtype=dtype)
        y = y.to(device=device, dtype=torch.long)

        # Forward pass: compute scores and loss
        scores = model_fn(x, params)
        loss = F.cross_entropy(scores, y)

        # Backward pass: PyTorch figures out which Tensors in the computational
        # graph has requires_grad=True and uses backpropagation to compute the
        # gradient of the loss with respect to these Tensors, and stores the
        # gradients in the .grad attribute of each Tensor.
        loss.backward()

        # Update parameters. We don't want to backpropagate through the
        # parameter updates, so we scope the updates under a torch.no_grad()
        # context manager to prevent a computational graph from being built.
        with torch.no_grad():
            for w in params:
                w -= learning_rate * w.grad

            # Manually zero the gradients after running the backward pass
            w.grad.zero_()

        if t % print_every == 0:
            print('Iteration %d, loss = %.4f' % (t, loss.item()))
            check_accuracy_part2(loader_val, model_fn, params)
            print()

```

✓ BareBones PyTorch: Train a Two-Layer Network

Now we are ready to run the training loop. We need to explicitly allocate tensors for the fully connected weights, `w1` and `w2`.

Each minibatch of CIFAR has 64 examples, so the tensor shape is `[64, 3, 32, 32]`.

After flattening, `x` shape should be `[64, 3 * 32 * 32]`. This will be the size of the first dimension of `w1`. The second dimension of `w1` is the hidden layer size, which will also be the first dimension of `w2`.

Finally, the output of the network is a 10-dimensional vector that represents the probability distribution over 10 classes.

You do not need to tune any hyperparameters. After training for one epoch, you should observe that the model learns non-trivial structure in the data.

```
hidden_layer_size = 4000
learning_rate = 1e-2

w1 = random_weight((3 * 32 * 32, hidden_layer_size))
w2 = random_weight((hidden_layer_size, 10))

train_part2(two_layer_fc, [w1, w2], learning_rate)
```

```
Iteration 0, loss = 3.4059
Checking accuracy on the val set
Got 165 / 1000 correct (16.50%)
```

```
Iteration 100, loss = 2.1259
Checking accuracy on the val set
Got 317 / 1000 correct (31.70%)
```

```
Iteration 200, loss = 1.8702
Checking accuracy on the val set
Got 351 / 1000 correct (35.10%)
```

```
Iteration 300, loss = 2.0990
Checking accuracy on the val set
Got 413 / 1000 correct (41.30%)
```

```
Iteration 400, loss = 2.1022
Checking accuracy on the val set
Got 389 / 1000 correct (38.90%)
```

```
Iteration 500, loss = 1.7296
Checking accuracy on the val set
Got 391 / 1000 correct (39.10%)
```

```
Iteration 600, loss = 1.7698
Checking accuracy on the val set
Got 448 / 1000 correct (44.80%)
```

```
Iteration 700, loss = 1.5263
Checking accuracy on the val set
Got 460 / 1000 correct (46.00%)
```

✓ Part III. PyTorch Module API

Barebone PyTorch requires that we track all the parameter tensors by hand. This is fine for small networks with a few tensors, but it would be extremely inconvenient and error-prone to track tens or hundreds of tensors in larger networks.

PyTorch provides the `nn.Module` API for you to define arbitrary network architectures, while tracking every learnable parameters for you. In Part II, we implemented SGD ourselves. PyTorch also provides the `torch.optim` package that implements all the common optimizers, such as RMSProp, Adagrad, and Adam. It even supports approximate second-order methods like L-BFGS! You can refer to the [doc](#) for the exact specifications of each optimizer.

To use the Module API, follow the steps below:

1. Subclass `nn.Module`. Give your network class an intuitive name like `TwoLayerFC`.
2. In the constructor `__init__()`, define all the layers you need as class attributes. Layer objects like `nn.Linear` are themselves `nn.Module` subclasses and contain learnable parameters, so that you don't have to instantiate the raw tensors yourself. `nn.Module` will track these internal parameters for you. Refer to the [doc](#) to learn more about the dozens of builtin layers. **Warning:** don't forget to call the `super().__init__()` first!
3. In the `forward()` method, define the *connectivity* of your network. You should use the attributes defined in `__init__` as function calls that take tensor as input and output the "transformed" tensor. Do *not* create any new layers with learnable parameters in `forward()`! All of them must be declared upfront in `__init__`.

After you define your Module subclass, you can instantiate it as an object and call it just like the NN forward function in part II.

Module API: Two-Layer Network

Here is a concrete example of a 2-layer fully connected network:

```
class TwoLayerFC(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super().__init__()
        # assign layer objects to class attributes
        self.fc1 = nn.Linear(input_size, hidden_size)
        # nn.init package contains convenient initialization methods
        # http://pytorch.org/docs/master/nn.html#torch-nn-init
        nn.init.kaiming_normal_(self.fc1.weight)
        self.fc2 = nn.Linear(hidden_size, num_classes)
        nn.init.kaiming_normal_(self.fc2.weight)

    def forward(self, x):
        # forward always defines connectivity
        x = flatten(x)
        scores = self.fc2(F.relu(self.fc1(x)))
        return scores

def test_TwoLayerFC():
    input_size = 50
    x = torch.zeros((64, input_size), dtype=dtype) # minibatch size 64, feature dimension 50
    model = TwoLayerFC(input_size, 42, 10)
    scores = model(x)
    print(scores.size()) # you should see [64, 10]
test_TwoLayerFC()

torch.Size([64, 10])
```

Module API: Check Accuracy

Given the validation or test set, we can check the classification accuracy of a neural network.

This version is slightly different from the one in part II. You don't manually pass in the parameters anymore.

```
def check_accuracy_part34(loader, model):
    if loader.dataset.train:
        print('Checking accuracy on validation set')
    else:
        print('Checking accuracy on test set')
    num_correct = 0
    num_samples = 0
    model.eval() # set model to evaluation mode
    with torch.no_grad():
        for x, y in loader:
            x = x.to(device=device, dtype=dtype) # move to device, e.g. GPU
```



```

y = y.to(device=device, dtype=torch.long)
scores = model(x)
_, preds = scores.max(1)
num_correct += (preds == y).sum()
num_samples += preds.size(0)
acc = float(num_correct) / num_samples
print('Got %d / %d correct (%.2f)' % (num_correct, num_samples, 100 * acc))

```

✓ Module API: Training Loop

We also use a slightly different training loop. Rather than updating the values of the weights ourselves, we use an Optimizer object from the `torch.optim` package, which abstract the notion of an optimization algorithm and provides implementations of most of the algorithms commonly used to optimize neural networks.

```

def train_part34(model, optimizer, epochs=1):
    """
    Train a model on CIFAR-10 using the PyTorch Module API.

    Inputs:
    - model: A PyTorch Module giving the model to train.
    - optimizer: An Optimizer object we will use to train the model
    - epochs: (Optional) A Python integer giving the number of epochs to train for

    Returns: Nothing, but prints model accuracies during training.
    """
    model = model.to(device=device) # move the model parameters to CPU/GPU
    for e in range(epochs):
        for t, (x, y) in enumerate(loader_train):
            model.train() # put model to training mode
            x = x.to(device=device, dtype=dtype) # move to device, e.g. GPU
            y = y.to(device=device, dtype=torch.long)

            scores = model(x)
            loss = F.cross_entropy(scores, y)

            # Zero out all of the gradients for the variables which the optimizer
            # will update.
            optimizer.zero_grad()

            # This is the backwards pass: compute the gradient of the loss with
            # respect to each parameter of the model.
            loss.backward()

            # Actually update the parameters of the model using the gradients
            # computed by the backwards pass.
            optimizer.step()

            if t % print_every == 0:
                print('Iteration %d, loss = %.4f' % (t, loss.item()))
                check_accuracy_part34(loader_val, model)
                print()

```

✓ Module API: Train a Two-Layer Network

Now we are ready to run the training loop. In contrast to part II, we don't explicitly allocate parameter tensors anymore.

Simply pass the input size, hidden layer size, and number of classes (i.e. output size) to the constructor of `TwoLayerFC`.

You also need to define an optimizer that tracks all the learnable parameters inside `TwoLayerFC`.

You do not need to tune any hyperparameters. After training for one epoch, you should observe that the model learns non-trivial structure in the data.

```
hidden_layer_size = 4000
learning_rate = 1e-2
model = TwoLayerFC(3 * 32 * 32, hidden_layer_size, 10)
optimizer = optim.SGD(model.parameters(), lr=learning_rate)

train_part34(model, optimizer)
```

```
Iteration 0, loss = 3.4457
Checking accuracy on validation set
Got 117 / 1000 correct (11.70)
```

```
Iteration 100, loss = 1.6623
Checking accuracy on validation set
Got 324 / 1000 correct (32.40)
```

```
Iteration 200, loss = 1.6656
Checking accuracy on validation set
Got 406 / 1000 correct (40.60)
```

```
Iteration 300, loss = 1.9933
Checking accuracy on validation set
Got 440 / 1000 correct (44.00)
```

```
Iteration 400, loss = 1.9508
Checking accuracy on validation set
Got 378 / 1000 correct (37.80)
```

```
Iteration 500, loss = 1.8041
Checking accuracy on validation set
Got 395 / 1000 correct (39.50)
```

```
Iteration 600, loss = 1.7404
Checking accuracy on validation set
Got 402 / 1000 correct (40.20)
```

```
Iteration 700, loss = 1.6593
Checking accuracy on validation set
Got 440 / 1000 correct (44.00)
```

✓ Part IV. PyTorch Sequential API

Part III introduced the PyTorch Module API, which allows you to define arbitrary learnable layers and their connectivity.

For simple models like a stack of feed forward layers, you still need to go through 3 steps: subclass `nn.Module`, assign layers to class attributes in `__init__`, and call each layer one by one in `forward()`. Is there a more convenient way?

Fortunately, PyTorch provides a container Module called `nn.Sequential`, which merges the above steps into one. It is not as flexible as `nn.Module`, because you cannot specify more complex topology than a feed-forward stack, but it's good enough for many use cases.

Sequential API: Two-Layer Network

Let's see how to rewrite our two-layer fully connected network example with `nn.Sequential`, and train it using the training loop defined above.

Again, you do not need to tune any hyperparameters. After one epoch of training, you should observe that the model learns non-trivial structure in the data.

```
# We need to wrap `flatten` function in a module in order to stack it
# in nn.Sequential
class Flatten(nn.Module):
    def forward(self, x):
        return flatten(x)

hidden_layer_size = 4000
learning_rate = 1e-2

model = nn.Sequential(
    Flatten(),
    nn.Linear(3 * 32 * 32, hidden_layer_size),
    nn.ReLU(),
    nn.Linear(hidden_layer_size, 10),
)

# you can use Nesterov momentum in optim.SGD
optimizer = optim.SGD(model.parameters(), lr=learning_rate,
                       momentum=0.9, nesterov=True)

train_part34(model, optimizer)
```

```
Iteration 0, loss = 2.2978
Checking accuracy on validation set
Got 153 / 1000 correct (15.30)

Iteration 100, loss = 1.6106
Checking accuracy on validation set
Got 401 / 1000 correct (40.10)

Iteration 200, loss = 1.9456
Checking accuracy on validation set
Got 392 / 1000 correct (39.20)

Iteration 300, loss = 2.0143
Checking accuracy on validation set
Got 401 / 1000 correct (40.10)

Iteration 400, loss = 1.7382
Checking accuracy on validation set
Got 452 / 1000 correct (45.20)

Iteration 500, loss = 1.6557
Checking accuracy on validation set
Got 413 / 1000 correct (41.30)

Iteration 600, loss = 1.7669
Checking accuracy on validation set
Got 435 / 1000 correct (43.50)

Iteration 700, loss = 1.8055
Checking accuracy on validation set
Got 461 / 1000 correct (46.10)
```

✓ Part V. CIFAR-10 open-ended challenge

In this section, you can experiment with whatever architecture you'd like on CIFAR-10.

Now it's your job to experiment with architectures, hyperparameters, loss functions, and optimizers to train a model that achieves high accuracy on the CIFAR-10 **validation** set within 10 epochs. You can use the `check_accuracy` and `train` functions from above. You can use either `nn.Module` or `nn.Sequential` API.

Describe what you did at Overleaf report.

- Layers in torch.nn package: <http://pytorch.org/docs/stable/nn.html>
- Activations: <http://pytorch.org/docs/stable/nn.html#non-linear-activations>
- Loss functions: <http://pytorch.org/docs/stable/nn.html#loss-functions>

- Optimizers: <http://pytorch.org/docs/stable/optim.html>

Things you might try:

- **Batch normalization:** Try adding batch normalization after fully-connected layers. Does training become more stable or faster?
- **Network architecture:** The network above has two layers of trainable parameters. Can you do better with a deep network?
- **Regularization:** Add l2 weight regularization, or perhaps use Dropout.

Tips for training

For each network architecture that you try, you should tune the learning rate and other hyperparameters. When doing this there are a couple important things to keep in mind:

- If the parameters are working well, you should see improvement within a few hundred iterations
- Remember the coarse-to-fine approach for hyperparameter tuning: start by testing a large range of hyperparameters for just a few training iterations to find the combinations of parameters that are working at all.
- Once you have found some sets of parameters that seem to work, search more finely around these parameters. You may need to train for more epochs.
- You should use the validation set for hyperparameter search, and save your test set for evaluating your architecture on the best parameters as selected by the validation set.

Going above and beyond

If you are feeling adventurous there are many other features you can implement to try and improve your performance. You are **not required** to implement any of these, but don't miss the fun if you have time!

- Alternative optimizers: you can try Adam, Adagrad, RMSprop, etc.
- Alternative activation functions such as leaky ReLU, parametric ReLU, ELU, or MaxOut.
- Model ensembles
- Data augmentation
- New Architectures
 - [ResNets](#) where the input from the previous layer is added to the output.
 - [DenseNets](#) where inputs into previous layers are concatenated together.
 - [This blog has an in-depth overview](#)
 - [LeNet-5](#) where convolutional layers and pooling are used to exploit spatial structure in images.
 - [AlexNet](#) which demonstrated the effectiveness of deep convolutional neural networks for large-scale image classification.
 - [This blog provides an accessible introduction to CNNs](#) explaining convolution, pooling, and why CNNs outperform fully-connected networks on image data.

Have fun and happy training!

✓ TO DO

✓ The model architecture, Optimization choices, Regularization

```
# TODO: #  
# Experiment with any architectures, optimizers, and hyperparameters. #  
# Achieve high accuracy on the *validation set* within 10 epochs. #
```

```

input_size = 3 * 32 * 32 # CIFAR-10 images are 3x32x32 pixels
hidden_1 = 1024
hidden_2 = 512
hidden_3 = 256
num_classes = 10

learning_rate = 1e-3
dropout_rate = 0.3
l2_regularization = 1e-4

```

MODEL ARCHITECTURE

```

# Building a Deep Fully Connected Neural Network using Sequential API
my_deep_model = nn.Sequential(
    # Flatten the 3D image tensor into a 1D vector
    Flatten(),

    # --- First Hidden Layer ---
    nn.Linear(input_size, hidden_1),
    nn.BatchNorm1d(hidden_1),      # Batch Normalization for faster, stable training
    nn.ReLU(),                    # Non-linear activation function
    nn.Dropout(dropout_rate),     # Dropout for regularization (prevents overfitting)

    # --- Second Hidden Layer ---
    nn.Linear(hidden_1, hidden_2),
    nn.BatchNorm1d(hidden_2),
    nn.ReLU(),
    nn.Dropout(dropout_rate),

    # --- Third Hidden Layer ---
    nn.Linear(hidden_2, hidden_3),
    nn.BatchNorm1d(hidden_3),
    nn.ReLU(),
    nn.Dropout(dropout_rate),

    # --- Output Layer ---
    # No activation or dropout at the end (CrossEntropyLoss expects raw scores)
    nn.Linear(hidden_3, num_classes)
)

```

OPTIMIZATION & REGULARIZATION

```

# Using Adam optimizer which generally performs better and faster than SGD.
# 'weight_decay' adds L2 regularization to further prevent overfitting.
my_optimizer = optim.Adam(
    my_deep_model.parameters(),
    lr=learning_rate,
    weight_decay=l2_regularization
)

```

TRAINING EXECUTION

```

# We assign our model and optimizer to the variables expected by the notebook
model = my_deep_model
optimizer = my_optimizer

```

✓ hyperparameters

HYPERPARAMETER SEARCH SPACE

```

# We define a small grid of hyperparameters to search over, avoiding "guessing"
learning_rates = [1e-3, 5e-4]
dropout_rates = [0.2, 0.4]

best_val_acc = 0.0
best_model = None
best_optimizer = None
best_hparams = {}

# A helper function to evaluate accuracy programmatically
def evaluate_accuracy(model, loader):
    model.eval()
    num_correct = 0
    num_samples = 0
    with torch.no_grad():
        for x, y in loader:
            x = x.to(device=device, dtype=dtype)
            y = y.to(device=device, dtype=torch.long)
            scores = model(x)
            preds = scores.max(1)[1]
            num_correct += (preds == y).sum().item()
            num_samples += preds.size(0)
    return float(num_correct) / num_samples

```

GRID SEARCH & MODEL TRAINING

```

for lr in learning_rates:
    for drop_rate in dropout_rates:
        print(f"\n--- Testing Configuration: LR={lr}, Dropout={drop_rate} ---")

        # Build the model architecture
        current_model = nn.Sequential(
            Flatten(),

            nn.Linear(3 * 32 * 32, 1024),
            nn.BatchNorm1d(1024),
            nn.ReLU(),
            nn.Dropout(drop_rate),

            nn.Linear(1024, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Dropout(drop_rate),

            nn.Linear(512, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Dropout(drop_rate),

            nn.Linear(256, 10)
        ).to(device)

        # Define the optimizer with L2 regularization (weight_decay)
        current_optimizer = optim.Adam(
            current_model.parameters(),
            lr=lr,
            weight_decay=1e-4
        )

        # Train this configuration for 3 epochs (search phase)
        train_part34(current_model, current_optimizer, epochs=3)

        # Check validation accuracy
        val_acc = evaluate_accuracy(current_model, loader_val)
        print(f"Validation Accuracy for LR={lr}, Dropout={drop_rate}: {val_acc * 100:.2f}%\n")

        # Save the best model

```

```
if val_acc > best_val_acc:
    best_val_acc = val_acc
    best_model = current_model
    best_optimizer = current_optimizer
    best_hparams = {'lr': lr, 'dropout': drop_rate}
```

```
Iteration 500, loss = 1.5452
Checking accuracy on validation set
Got 532 / 1000 correct (53.20)
```

```
Iteration 600, loss = 1.3517
Checking accuracy on validation set
Got 525 / 1000 correct (52.50)
```

```
Iteration 700, loss = 1.1761
Checking accuracy on validation set
Got 519 / 1000 correct (51.90)
```

Validation Accuracy for LR=0.0005, Dropout=0.2: 53.30%

--- Testing Configuration: LR=0.0005, Dropout=0.4 ---

```
Iteration 0, loss = 2.3380
Checking accuracy on validation set
Got 118 / 1000 correct (11.80)
```

```
Iteration 100, loss = 1.9771
Checking accuracy on validation set
Got 363 / 1000 correct (36.30)
```

```
Iteration 200, loss = 1.9451
Checking accuracy on validation set
Got 384 / 1000 correct (38.40)
```

```
Iteration 300, loss = 1.6688
Checking accuracy on validation set
Got 399 / 1000 correct (39.90)
```

```
Iteration 400, loss = 1.7017
Checking accuracy on validation set
Got 431 / 1000 correct (43.10)
```

```
Iteration 500, loss = 1.6274
Checking accuracy on validation set
Got 438 / 1000 correct (43.80)
```

```
Iteration 600, loss = 1.6103
Checking accuracy on validation set
Got 436 / 1000 correct (43.60)
```

```
Iteration 700, loss = 1.9037
Checking accuracy on validation set
Got 464 / 1000 correct (46.40)
```

```
Iteration 0, loss = 1.7382
Checking accuracy on validation set
Got 467 / 1000 correct (46.70)
```

```
Iteration 100, loss = 1.5183
Checking accuracy on validation set
Got 462 / 1000 correct (46.20)
```

```
Iteration 200, loss = 1.3303
Checking accuracy on validation set
```

FINAL CONFIGURATION SELECTION

```
print("=====")
print("Winning configuration found")
print(f"Best Validation Accuracy: {best_val_acc * 100:.2f}%")
print(f"Best Hyperparameters: {best_hparams}")
print("=====")
```

```
# Assign the winning model and optimizer to the variables expected by the notebook
model = best_model
optimizer = best_optimizer
```

```
=====
Winning configuration found
Best Validation Accuracy: 53.30%
Best Hyperparameters: {'lr': 0.0005, 'dropout': 0.2}
=====
```

Part V. CIFAR-10 open-ended challenge

```
train_part34(model, optimizer, epochs=10)
```

```
Got 578 / 1000 correct (57.80)
```

```
Iteration 700, loss = 1.2505
Checking accuracy on validation set
Got 584 / 1000 correct (58.40)
```

```
Iteration 0, loss = 1.0130
Checking accuracy on validation set
Got 575 / 1000 correct (57.50)
```

```
Iteration 100, loss = 1.4577
Checking accuracy on validation set
Got 580 / 1000 correct (58.00)
```

```
Iteration 200, loss = 0.9080
Checking accuracy on validation set
Got 560 / 1000 correct (56.00)
```

```
Iteration 300, loss = 0.9102
Checking accuracy on validation set
Got 567 / 1000 correct (56.70)
```

```
Iteration 400, loss = 1.4997
Checking accuracy on validation set
Got 555 / 1000 correct (55.50)
```

```
Iteration 500, loss = 1.2648
Checking accuracy on validation set
Got 568 / 1000 correct (56.80)
```

```
Iteration 600, loss = 0.9387
Checking accuracy on validation set
Got 573 / 1000 correct (57.30)
```

```
Iteration 700, loss = 1.0029
Checking accuracy on validation set
Got 575 / 1000 correct (57.50)
```

```
Iteration 0, loss = 0.8699
Checking accuracy on validation set
Got 583 / 1000 correct (58.30)
```

```
Iteration 100, loss = 1.1544
Checking accuracy on validation set
Got 571 / 1000 correct (57.10)
```

```
Iteration 200, loss = 0.9225
Checking accuracy on validation set
Got 580 / 1000 correct (58.00)
```

```
Iteration 300, loss = 1.1663
Checking accuracy on validation set
Got 587 / 1000 correct (58.70)
```

```
Iteration 400, loss = 1.2665
Checking accuracy on validation set
Got 580 / 1000 correct (58.00)
```

```
Iteration 500, loss = 0.7779
```


✓ Test set -- run this only once

Now that we've gotten a result we're happy with, we test our final model on the test set (which you should store in `best_model`). Think about how this compares to your validation set accuracy.

```
best_model = model  
check_accuracy_part34(loader_test, best_model)
```

```
Checking accuracy on test set  
Got 5740 / 10000 correct (57.40)
```

Start coding or [generate](#) with AI.